Common Datasets in Spark

Here, we will discuss common datasets that Spark handles, including real-world applications. We'll also explore the key Spark data abstractions: RDDs and DataFrames, along with usage examples and syntax explanations.

Learning Objectives

By the end of this train, you should be able to:

- Describe datasets commonly used in Spark.
- Explain the difference between RDDs and DataFrames and their use cases.
- Use PySpark to work with a variety of datasets in real-world scenarios.

Resilient Distributed Datasets (RDDs)

RDDs are the most fundamental data structure in Spark, representing a fault-tolerant and distributed collection of elements. They provide low-level control over data and are used when you need fine-grained transformations or working with unstructured data.

```
rdd = spark.sparkContext.parallelize([("John", 28), ("Jane", 35)])
rdd.collect()
```

Use Case: Data cleansing in ETL jobs where transformation logic is complex and cannot be expressed via SQL/DataFrame syntax.

DataFrames

DataFrames are distributed collections of data organized into named columns. They are conceptually equivalent to a table in a relational database and are optimized using Spark's Catalyst optimizer and Tungsten execution engine.

```
from pyspark.sql import Row

df = spark.createDataFrame([
    Row(name="Alice", age=34),
    Row(name="Bob", age=45)
])
df.show()
```

Use Case: Performing analytics on structured logs, large tables or machine learning features in a scalable and SQL-friendly way.

CSV Datasets

Comma-Separated Values (CSV) are one of the most common file types for tabular data. They are easy to read and widely used for exporting and importing from Excel, databases, and legacy systems.

```
df_csv = spark.read.csv("people.csv", header=True, inferSchema=True)
df_csv.show()
```

Use Case: Loading tabular data from legacy exports or spreadsheets for transformation and analysis in Spark.

JSON Datasets

JSON (JavaScript Object Notation) is a widely used format for semi-structured data. Spark supports reading nested JSON structures and inferring schemas automatically.

```
df_json = spark.read.json("data.json")
df_json.printSchema()
```

Use Case: Reading logs, event streams, or API response data that is delivered in JSON format.

Parquet Datasets

Parquet is a columnar storage format highly optimised for analytics. It supports efficient compression and encoding schemes that save storage space and speed up query performance.

```
df_parquet = spark.read.parquet("data.parquet")
df_parquet.select("name").show()
```

Use Case: Big data analytical queries in data lakes or data warehouses such as AWS S3 or Delta Lake.

Avro Datasets

Avro is a row-based binary format designed for serialisation and deserialisation. It's often used with Apache Kafka and provides schema evolution support.

```
df_avro = spark.read.format("avro").load("data.avro")
df_avro.show()
```

Use Case: Ingesting data from Kafka or messaging systems where schema evolution and compression are critical.

ORC Datasets

Optimised Row Columnar (ORC) is a file format designed to store Hive data efficiently. It's well-suited for workloads with heavy read operations.

```
df_orc = spark.read.format("orc").load("data.orc")
df_orc.printSchema()
```

Use Case: Large-scale data warehousing in Hadoop-based ecosystems (Hive, HDFS).

JDBC Datasets

JDBC allows Spark to connect to relational databases using SQL. It's useful for accessing production systems and integrating traditional databases into Spark pipelines.

```
jdbc_url = "jdbc:postgresql://localhost:5432/mydb"
props = {{
    "user": "myuser",
    "password": "mypassword"
}}

df_jdbc = spark.read.jdbc(url=jdbc_url, table="public.users", properties=props)
df_jdbc.show()
```

Use Case: Reading directly from an OLTP database such as PostgreSQL or MySQL for near real-time reporting or transformations.

Conclusion

Spark provides powerful abstractions (RDDs and DataFrames) and supports a wide array of dataset formats to handle real-world data processing needs. Knowing which dataset format and abstraction to use improves efficiency, scalability, and ease of implementation in big data projects.

▼Key Takeaways ✖

- **Takeaway 1:** Spark supports a wide variety of dataset types, including structured, semi-structured, and binary formats.
- **Takeaway 2:** RDDs provide low-level data control, while DataFrames offer high-level abstraction and optimisation.
- **Takeaway 3:** Columnar formats like Parquet and ORC are best suited for analytics and large-scale data storage.
- **Takeaway 4:** JSON and Avro are ideal for streaming and semi-structured data.
- **Takeaway 5:** JDBC is ideal for integrating relational databases directly into Spark pipelines.